

Modern Application Development II

Hospital Management System V2 - Project Report

• Student Details

Name : Pritam Saha

Roll Number : 23F1002917

Student Email : 23f1002917@ds.study.iitm.ac.in

About : I love developing applications that simplify everyday life for people.

• Project Details

- **Project Title : Hospital Management System - V2**

- **Project Statement :**

HMS-V2 web application will allow efficient management of **appointments like booking, tracking, notifying and managing patients, doctors, and treatment records** in one unified place.

Full Project Statement Link -

https://docs.google.com/document/u/1/d/e/2PACX-1vQtRY8Yq_ST253dCUhYwxTS52N4vNCMoBpwB2yNnXEQDcNI_lkbr5yl5ne2j0ym5tcNT65on3Rfl_x-rB/pub

- **Approach :**

I have used JWT for role based authentication and authorization in my Flask backend for efficiently identify and manage different users (admin, patient, doctor) resources. And Vue Js for dynamically handling different users' views in my frontend.

• AI/LLM Declaration

- I used **ChatGPT (GPT - 5.2)** mainly in my **flask backend** to assist me with debugging different errors, then structuring my code and to implement python subprocess, setting up celery and redis and to learn adding class methods in my database model classes and to do simple searching queries. And in **frontend** to learn and implement axios interceptor for handling auto jwt token refresh and adding access token to each request.
- And used **Gemini (2.5 Flash)** mainly in my **vue frontend** to assist me with html, css and bootstrap styling and to integrate them into a vue SFC to properly render them. I am not used to javascript that much so I made the LLM evaluate my JS code to identify errors. In my **frontend** initial dashboard's layout is generated by the model because the bootstrap's default dashboard layout is not working properly with the vue js parent child view architecture. And to debug console errors on the browsers devtool.
- **Usage Percentages:**
 - ◆ Backend (Flask + DB) : **15%**
 - ◆ Frontend (HTML/JS) : **35%**
 - ◆ Styling/UI : **8%**
 - ◆ Auth + CRUD Logic : **10%**
 - ◆ Extras : **4%**

• Frameworks and Technologies Used

→ Backend :

- **Flask** : Python microweb framework used as backend
- **Flask-SQLAlchemy** : Flask extension for Object Relational Mapper for SQLite database
- **Flask-JWT-Extended** : Flask extension to implement token based authentication and authorization
- **Flask-Mail** : Flask extension to send email from the backend server
- **Flask-Restful** : Flask extension to implement REST APIs
- **Flask-Cors** : Flask extension to implement CORS policy for the application
- **Flask-Caching** : Flask extension to send email from the backend server
- **Jinja2** : Templating engine for HTML pages (used for doctor report creation only)
- **Sqlite3** : File based serverless relational database
- **Redis** : The in-memory database for caching and as a message broker
- **Celery** : It's a task queue with focus on real-time processing, while also supporting task scheduling
- **Redis-py** : Python client library for redis database interactions
- **Bcrypt** : Python library for password hashing
- **Email_validator** : Python library to validate emails in the backend

→ Frontend :

- **Vite** : Javascript project building and scaffolding tool
- **Vue3** : The Progressive JavaScript Framework for building web interfaces
- **Vue-Router** : Vue official extension for frontend application routing
- **Vue-Pinia** : Vue official extension for application state management
- **Vue3-Select-Component** : Vue3 extension for custom select input
- **Axios** : JavaScript library used to make HTTP requests
- **Bootstrap5** : CSS framework for styling and designing responsive UI
- **Bootstrap-Icons** : The collection of SVG icons package by bootstrap

• ER Diagrams

→ Login and Registration Schema: ([ER-diagram HMS-Login Schema.png](#))

- ◆ **User** – Stores user credentials (id, email, password, is_active, fs_uniquifier)
- ◆ **Role** – Stores the roles (id, name, description)
- ◆ **Roles_Users** – The joining table between role and user (id, user_id, role_id)
- ◆ **User_Tokens** – Stores all JWT generated for user (id, user_id, jti, type, create_datetime, expiry_datetime, valid)
- ◆ **Patient** – Stores the patient user data (id, user_id, public_id, full_name, contact, dob, gender, height_cm, weight_kg)
- ◆ **Doctor** – Stores the doctor user data (id, user_id, public_id, full_name, gender, license, qualification, experience, description, contact)
- **Relationships:**
 - **One-to-One** → **User – Patient**
 - **One-to-One** → **User – Doctor**
 - **One-to-Many** → **User → User_Tokens**
 - **Many-to-Many** → **User → Roles_Users ← Role**

→ **Patient Schema:** ([ER-diagram_HMS-Patient_Schema.png](#))

- ◆ **Appointment** – Stores the patient and doctors appointment schedules (id, patient_id, doctor_id, date, start_time, end_time, status)
 - ◆ **Treatment** – Stores the patient treatments for an appointment (id, appointment_id, visit_type, tests_done, diagnosis, prescription, medicine, notes)
 - ◆ **Patient** – Stores the patient user data (id, user_id, public_id, full_name, contact, dob, gender, height_cm, weight_kg)
 - ◆ **Doctor** – Stores the doctor user data (id, user_id, public_id, full_name, gender, license, qualification, experience, description, contact)
- **Relationships:**
 - **One-to-One** → **Treatment – Appointment**
 - **One-to-Many** → **Patient → Appointment**
 - **One-to-Many** → **Doctor → Appointment**

→ **Doctor Schema:** ([ER-diagram_HMS-Doctor_Schema.png](#))

- ◆ **Availability** – Stores the doctor availability (id, doctor_id, slot_id, date, start_time, end_time, diagnosis_limit, appointment_count, status)
 - ◆ **Department** – Stores the departments (id, name, description, type)
 - ◆ **Departments_Doctors** – The joining table between department and doctor (id, doctor_id, department_id)
 - ◆ **Specialization** – Stores all specializations (id, name)
 - ◆ **Doctor** – Stores the doctor user data (id, user_id, public_id, full_name, gender, license, qualification, experience, description, contact)
- **Relationships:**
 - **One-to-Many** → **Doctor → Availability**
 - **One-to-Many** → **Specialization → Doctor**
 - **Many-to-Many** → **Doctor → Departments_Doctors ← Department**

→ **Notification Schema:** ([ER-diagram_HMS-Notification_Schema.png](#))

- ◆ **Notification** – Stores the user notifications (id, user_id, date, time, data, type)
 - ◆ **User** – Stores user credentials (id, email, password, is_active, fs_uniquifier)
- **Relationships:**
 - **One-to-Many** → **User → Notification**

● **API Resource Endpoints**

→ **Login And Registration APIs:**

<code>/api</code>	<code>GET</code>	<code>API index</code>
<code>/api/register/patient</code>	<code>POST</code>	<code>Registers Patient</code>
<code>/api/register/doctor</code>	<code>POST</code>	<code>Registers Doctor</code>
<code>/api/login</code>	<code>POST</code>	<code>Login the User</code>
<code>/api/token/refresh</code>	<code>GET</code>	<code>Refreshes Tokens</code>
<code>/api/token/user/role-valid</code>	<code>POST</code>	<code>Checks the token validity</code>

/api/logout	POST	Logout the User
/api/logout/all	POST	Logout the User from everywhere

→ Admin APIs:

/api/dashboard/admin	GET	Gets the admin data
/api/dashboard/admin/appointments	GET, PATCH	All appointments management
/api/dashboard/admin/patients	GET	Gets all the patient
/api/dashboard/admin/patient-appointments	GET	Gets the patient appointment
/api/dashboard/admin/patient-treatment	GET	Gets the patient treatment
/api/dashboard/admin/doctors	GET	Gets all the doctor
/api/dashboard/admin/stats	GET	Gets the stats
/api/dashboard/admin/specializations	GET	Gets the specialization
/api/dashboard/admin/doctor	GET, POST, PATCH, DELETE	Manages the doctors
/api/dashboard/admin/patient	GET, POST, PATCH, DELETE	Manages the patients
/api/dashboard/admin/notifications	GET, DELETE	Manages the notification
/api/dashboard/admin/patient	GET, POST, PATCH, DELETE	Manages the patients
/api/dashboard/admin/appointments/search	GET	Searches the appointments
/api/dashboard/admin/patients/search	GET	Searches the patients
/api/dashboard/admin/doctors/search	GET	Searches the doctors

→ Doctor APIs:

/api/dashboard/doctor	GET	Gets the doctor data
/api/dashboard/doctor/availability	GET, PATCH	Manages doctor availability
/api/dashboard/doctor/appointments	GET, PATCH	Manages doctor appointments
/api/dashboard/doctor/week-stats	GET	Gets the week stats
/api/dashboard/doctor/assigned-patients	GET	Gets the assigned patients
/api/dashboard/doctor/patient-history	GET, POST, PATCH	Manages the patient history

<code>/api/dashboard/doctor/notifications</code>	GET, DELETE	Manages the notifications
<code>/api/dashboard/doctor/assigned-patients/search</code>	GET	Searches the patient
<code>/api/dashboard/doctor/appointments/search</code>	GET	Searches the appointments

→ **Patient APIs:**

<code>/api/dashboard/patient</code>	GET	Gets the patient data
<code>/api/dashboard/patient/doctor-list</code>	GET	Gets the department doctors
<code>/api/dashboard/patient/upcoming-appointments</code>	GET, PATCH	Manages upcoming appointments
<code>/api/dashboard/patient/departments</code>	GET	Gets the department
<code>/api/dashboard/patient/appointment-history</code>	GET	Gets the patient appointments
<code>/api/dashboard/patient/appointment-treatment</code>	GET	Gets the patient treatment
<code>/api/dashboard/patient/book-appointment</code>	GET, POST	Manages the appointment booking
<code>/api/dashboard/patient/notifications</code>	GET, DELETE	Manages the notifications
<code>/api/dashboard/patient/upcoming-appointments/search</code>	GET	Searches the upcoming appointments
<code>/api/dashboard/patient/departments/search</code>	GET	Searches the departments
<code>/api/dashboard/patient/doctor-list/search</code>	GET	Searches the doctor list
<code>/api/dashboard/patient/appointment-history/search</code>	GET	Searches the appointment history
<code>/api/dashboard/patient/export-csv</code>	GET, POST	Requests for report export
<code>/api/dashboard/patient/download-csv</code>	GET	Downloads the CSV report

→ API documentation link - [HMS V2 API Documentation.yaml](#)

- **Presentation Video Link**

→ Google Drive link to the video -

<https://drive.google.com/file/d/1gltJ6L7xiDPZXtatUgZjUXDaGR3oIGvE/view?usp=sharing>